Proceedings of the 90th Annual International Meeting of the Psychometric Society, Minneapolis, United States (2025).

Hyperparameter Tuning

0.1 Regularization

The package `glmnet` v4.1-8 (Friedman et al., 2010) was used to fit all regularized regression models and the hyperparameter lambda was tuned using the default settings in the built-in function cv.glmnet using 10-fold cross validation. The family argument was specified as binomial, implying that the model was refit for each value of lambda using iteratively reweighted least squares (IRLS) estimation. For elastic net, a second hyperparameter, α – the L1 ratio– was tuned using a manual search over 11 candidate values (0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0) and cv.glmnet() was used to tune λ for each alpha tuning as the cross validation function does not allow for tuning α directly. The α value with the smallest corresponding cross-validated error, or binomial deviance, was 0.70 and largest λ value with an associated error within 1 standard error of the minimum cross validation error was .00069.

0.2 Trees

The packages `caret` v7.0-1 (Kuhn, 2008) and `rpart` v4.1.23 (Therneau & Atkinson, 2023) were used to train classification trees to predict network ties using recursive binary splitting. At the default settings, node purity was assessed using gini splitting index, the vector of prior probabilities was set to be proportional to the number of ties, and the off diagonals in the loss matrix were set to 1—implying that false positives and negatives were penalized equally. The minimum number of observations per node for attempting a split, *minsplit*, was left at the default value of n = 20. The complexity parameter, *cp*, was tuned over a user supplied sequence of 50 values ranging from 0.001 to 0.05, with equally spaced 0.001 intervals. A lower bound of .001 was chosen to avoid overfitting because deeper trees tend to perform worse in the test dataset. The complexity parameter sets a threshold for each split to improve the model by at least a certain amount for a split to be retained, allowing for pruning the tree of any unnecessary branches. The tuned value for *cp* was .001.

0.3 Ensemble Methods

R package `randomForest` v4.7-1.2 (Liaw & Wiener, 2002) was used to train the bagging and random forest algorithms.

0.3.1 Bagging

The number of trees grown, *n_tree*, were chosen among the values 100, 250, 300, 500, and 750 trees via 10-fold cross-validation. This process optimized out of bag classification error based on majority vote (across bags), whereas each tree in a given bag was trained to minimize node impurity, calculated using Gini index. The results of cross-validation supported the default value of 500 and the final model was run using default settings.

0.3.2 Random Forest

R packages `randomForest` v4.7-1.2 (Liaw & Wiener, 2002) and `caret` v7.0-1 (Kuhn, 2008) were used to fit the random forest model. The number of trees grown was fixed at 500 and the number of predictors sampled at each split, *mtry*, was tuned over 7 candidate values ranging from 3 to 9, reflecting the fact that these data contained only a handful of strongly predictive covariates. The

default value for this parameter, which is the square root of the total number of features ($\sqrt{29}$ = 5 in this dataset), was very close to the value of 6 chosen through cross validation.

0.3.3 XGBoost

Gradient boosting with decision trees was implemented using `xgboost` v1.7.8.1 (Chen et al., 2024) in conjunction with `ParBayesianOptimization` v1.2.6 (Wilson, 2022). The latter was used to tune eight hyperparameters to minimize logistic loss via 10-fold cross validation. The use of bayesian optimization algorithm for hyperparameter tuning tends to outperform conventional tuning methods (random and grid search; e.g., Putatunda and Rama, 2018). The tuning parameters and their bounds (in parentheses) were specified as follows: learning rate, *eta* (0 to 1); minimum loss reduction, *gamma* (0 to 100); upper bound for tree depth, *max_depth* (2 to 10); minimum sum of instance weight for further partitioning, *min_child_weight* (1 to 25); the proportion of training data used to build each tree, subsample (0.25 to 1); the fraction of features sampled to build each tree, *colsample_by_tree* (.1 to .9); the maximum number of rounds of no improvement before stopping training in the validation set, *early_stopping_rounds* (30 to 100); and the number of boosting iterations, *n_rounds* (300 to 1000).

References

Chen, T., He, T., Benesty, M., Khotilovich, V., Tang, Y., Cho, H., Chen, K., Mitchell, R., Cano, I., Zhou, T., Li, M., Xie, J., Lin, M., Geng, Y., Li, Y., & Yuan, J. (2024). *Xgboost: Extreme gradient boosting* [R package version 1.7.8.1]. https://CRAN.R-project.org/package=xgboost

Friedman, J. H., Hastie, T., & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, *33*(1), 1–22. https://doi.org/10.18637/jss.v033.i01

Kuhn, M. (2008). Building predictive models in r using the caret package. *Journal of Statistical Software*, *28*(5), 1–26. https://doi.org/10.18637/jss.v028.i05

Liaw, A., & Wiener, M. (2002). Classification and regression by randomforest. *R News*, *2*(3), 18–22. https://CRAN.R-project.org/doc/Rnews/

Putatunda, S., & Rama, K. (2018). A comparative analysis of hyperopt as against other approaches for hyper-parameter optimization of xgboost. *Proceedings of the 2018 International Conference on Signal Processing and Machine Learning*, 6–10. https://doi.org/10.1145/3297067.3297080

Therneau, T., & Atkinson, B. (2023). *Rpart: Recursive partitioning and regression trees* [R package version 4.1.23]. https://CRAN.R-project.org/package=rpart

Wilson, S. (2022). *Parbayesianoptimization: Parallel bayesian optimization of hyperparameters* [R package version 1.2.6]. https://CRAN.R-project.org/package=ParBayesianOptimization